

# דוח מסכם פרויקט רשתות: חלק 2 - יישום וניתוח תעבורה

שמות הסטודנטים: נועה צדוק 214396434, לירן שטרנברג 211895578 ונעם רחמים 212551121

תאריך: 07.01.2026

[קישור ל-GITHUB](#)

## 1. הסבר כללי על המערכת ומבנה הקוד

המערכת שפותחה היא אפליקציית צ'אט המדמה את אפליקציית WhatsApp, ופועלת בארכיטקטורת Client-Server (שרת-לקוח) על גבי פרוטוקול TCP. המערכת מאפשרת למספר משתמשים להתחבר במקביל לשרת מרכזי, לקבל רשימת משתמשים מחוברים בזמן אמת, ולנהל שיחות פרטיות עם ממשק גרפי מתקדם.

רכיבי המערכת:

א. צד השרת - server.py

השרת מהווה את "המוח" של המערכת ואחראי על ניהול החיבורים והעברת ההודעות.

תקשורת (Sockets) – השרת מאזין בפורט הקבוע 5566 ובכתובת 0.0.0.0 (מקבל תקשורת מכל ממשק רשת).

ריבוי תהליכים (Multi-Threading): כדי לעמוד בדרישה לתמוך ב-5 לקוחות לפחות בו-זמנית, השרת יוצר Thread (תהליכון) ייעודי לכל לקוח שמתחבר באמצעות הפונקציה handle\_client. כך השרת פנוי לקבל חיבורים נוספים כל הזמן.

ניהול משתמשים וסנכרון (Lock):

המערכת מנהלת מילון clients הממפה בין שם המשתמש ל-Socket שלו.

מכיוון שמשאב זה משותף לכל התהליכים, נעשה שימוש ב-threading.Lock. כל גישה למילון (הוספה, מחיקה, או בדיקת יעד) נעשית בתוך בלוק with lock, מה שמונע Race Conditions - התנגשות בין תהליכים המנסים לגשת לאותו משאב בו-זמנית - וקריסת נתונים.

פרוטוקול האפליקציה (Application Layer Protocol):

- התחברות: הלקוח שולח שם משתמש -> השרת בודק כפילות -> מחזיר APPROVE או REFUSED.
- עדכון רשימה: בעת שינוי בסטטוס משתמשים, השרת שולח הודעת LIST[user1,user2]... לכולם.
- העברת הודעה: הודעה נשלחת מהלקוח בפורמט target|message. השרת מנתב אותה ליעד בפורמט MSG|sender|content.

| סוג הודעה     | שולח -> מקבל        | פורמט             | תיאור                      |
|---------------|---------------------|-------------------|----------------------------|
| בקשת התחברות  | לקוח -> שרת         | Username          | שם המשתמש הראשוני          |
| אישור / סירוב | שרת -> לקוח         | APPROVE / REFUSED | תשובת השרת לבדיקת כפילויות |
| עדכון רשימה   | שרת -> כל הלקוחות   | LIST              | Abigail, Hanita...         |
| הודעת הצ'אט   | לקוח -> שרת -> לקוח | MSG               | sender                     |

ב. צד הלקוח (client\_gui.py)

הלקוח הוא ממשק המשתמש (GUI) המאפשר אינטראקציה נוחה.

ממשק גרפי (Tkinter): הממשק עוצב בסגנון "Dark Mode" המזכיר את WhatsApp Web, וכולל בועות הודעה מעוצבות, רשימת אנשי קשר דינמית ומונה הודעות שלא נקראו.

תקשורת אסינכרונית: כדי שהממשק הגרפי לא "יקפא" (Freeze) בזמן ההמתנה להודעה, הלקוח מריץ Thread נפרד (receive\_messages) שמאזין באופן רציף לתעבורה מהשרת ומעדכן את המסך בזמן אמת, בעוד ה-Main Loop של ה-GUI מטפל בלחיצות המשתמש.

## 2. הוראות התקנה והרצה

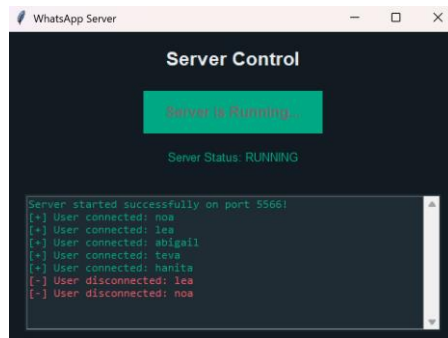
כדי להפעיל את המערכת בסביבת פיתוח מקומית:

דרישות קדם: וודא שמותקנת סביבת Python 3.x.

### הפעלת השרת:

- פתח מסוף (Terminal/CMD).
- הרץ את הפקודה: `python server.py`
- פלט צפוי: `Server running on 0.0.0.0:5566`....

GUI של השרת

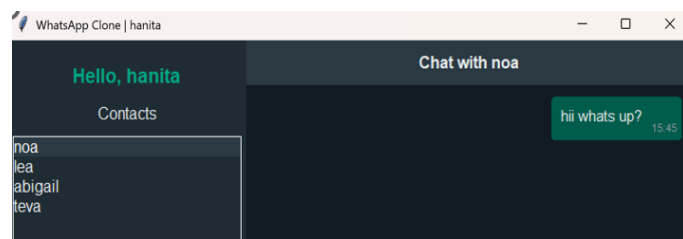


### הפעלת הלקוחות:

- פתח מסוף חדש והרץ: `python client_gui.py`
- בחלון הקופץ, הזן שם משתמש (למשל noa).
- פתח מסופים נוספים והתחבר עם שמות שונים (למשל lea, abigail).

## 3. דוגמאות קלט ופלט (GUI)

להלן תצוגת המערכת בזמן פעולה. ניתן לראות את רשימת אנשי הקשר בצד שמאל, ואת חלון הציאת עם בועות ההודעה (ירוק עבורי, אפור עבור הצד השני).



### שם השתמש המחובר לאותו חלון

תרחיש שיחה בין lea ל-noa. ניתן לראות את ההודעות "hi have a nice day" ו-"thank you" מוצגות בצורה ויזואלית תקינה

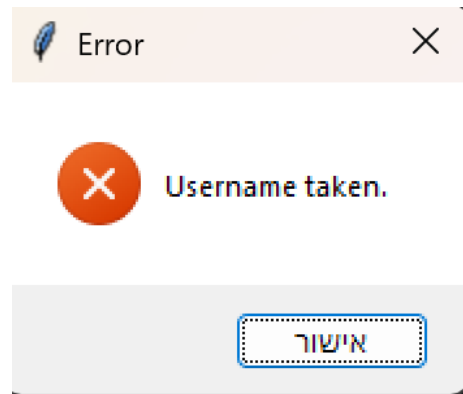
### תהליך שליחת הודעה:

קלט: המשתמש בוחר איש קשר ומקליד הודעה בתיבה התחתונה.

עיבוד: הלקוח אורז את ההודעה לפורמט `target|message` ושולח לשרת.

פלט: ההודעה מופיעה מיד במסך השולח כבועה ירוקה, ומגיעה לנמען כבועה אפורה.

במקרה והמשתמש רוצה להתחבר עם אותו שם המשתמש לא יהיה ניתן ע"פ דרישות הפרויקט.



### מה קורה בצד השרת?

**with lock:** - השרת נועל את הגישה למשתנים המשותפים כדי למנוע מצב ששני אנשים מתחברים באותה מאית שנייה עם אותו שם.

**if username in clients:** - זו הבדיקה עצמה. המשתנה clients הוא מילון שמחזיק את כל המחוברים. השרת בודק האם השם שהוקלד כבר קיים כמפתח במילון הזה.

**client\_socket.send("REFUSED".encode()):** - אם השם קיים, השרת שולח בחזרה ללקוח את המחרוזת "REFUSED".

**client\_socket.close():** - השרת מנתק את החיבור מיד.

### מה קורה בצד הלקוח?

כשהקוד מקבל את הסירוב מהשרת הוא מקפיץ את הודעת השגיאה:

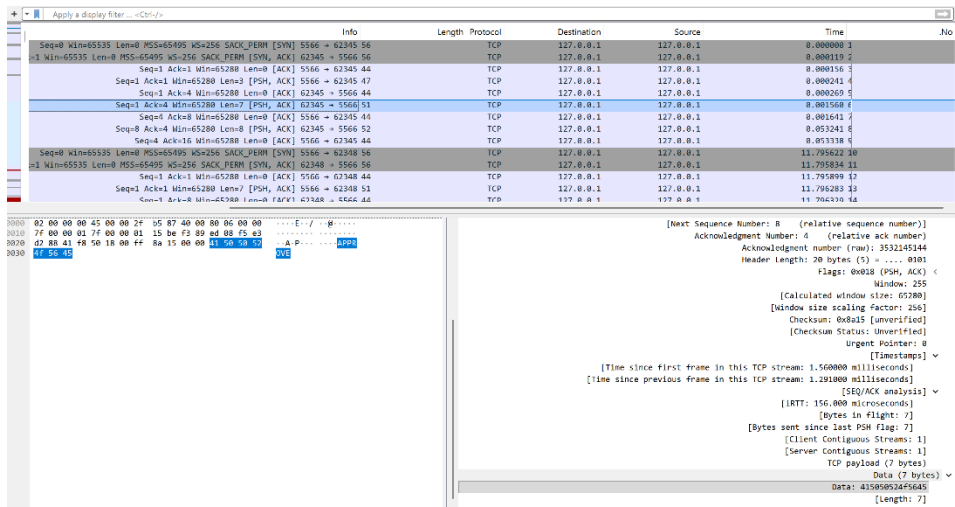
Username taken.

```
try:
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((SERVER_IP, PORT))
    client.send(input_user.encode())

    resp = client.recv(1024).decode()
    if resp == "APPROVE":
        username = input_user
    else:
        messagebox.showerror("Error", "Username taken.")
        client.close()
```

4. ניתוח תעבורה של היישום (Wireshark)

בפרק זה ננתח את התעבורה שנלכדה באמצעות Wireshark במחשק Adapter for loopback traffic capture (כתובת 127.0.0.1).



תוכן הפקטה (Payload): מה שמודגש בצד שמאל למטה, ניתן לראות בבירור את הטקסט "APPROVE". משמעות: זוהי הפקודה שהוגדרה בפרוטוקול האפליקציה.

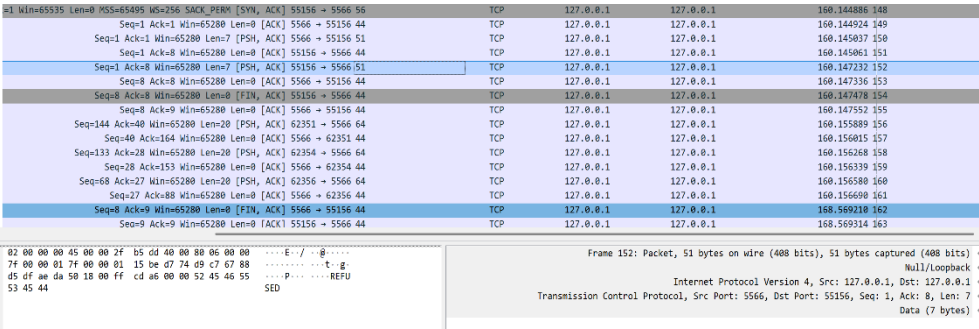
השרת בדק את שם המשתמש, מצא שהוא פנוי, ושלח אישור ללקוח כדי שיפתח את חלון הצ'אט הראשי.

הערה: בשדה CHECKSUM ההודעה 'Unverified' תקינה ונובעת מכך ש-Wireshark לוכד את המידע לפני שהמחשב מספיק לחשב את אימות הנתונים (Checksum). החישוב מתבצע ברגע האחרון ע"י החומרה כדי לחסוך משאבים, ולכן לא מופיע בלידה

לכידת תעבורת TCP בין פורט 5566 לפורטים אקראיים של הלקוח

|                                                                                |     |           |           |              |
|--------------------------------------------------------------------------------|-----|-----------|-----------|--------------|
| Seq=5566 Win=0 MSS=65535 WS=256 SACK_PERM [SYN, ACK] 5566 → 62345 56           | TCP | 127.0.0.1 | 127.0.0.1 | 0.000000 1   |
| Seq=5566 Win=65535 Len=0 MSS=65535 WS=256 SACK_PERM [PSH, ACK] 62345 → 5566 56 | TCP | 127.0.0.1 | 127.0.0.1 | 0.000119 2   |
| Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5566 → 62345 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 0.000156 3   |
| Seq=1 Ack=1 Win=65280 Len=0 [PSH, ACK] 5566 → 62345 47                         | TCP | 127.0.0.1 | 127.0.0.1 | 0.000241 4   |
| Seq=1 Ack=4 Win=65280 Len=0 [ACK] 62345 → 5566 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 0.000269 5   |
| Seq=1 Ack=8 Win=65280 Len=7 [PSH, ACK] 62345 → 5566 51                         | TCP | 127.0.0.1 | 127.0.0.1 | 0.001568 6   |
| Seq=1 Ack=8 Win=65280 Len=0 [ACK] 5566 → 62345 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 0.001641 7   |
| Seq=4 Ack=4 Win=65280 Len=0 [PSH, ACK] 62345 → 5566 52                         | TCP | 127.0.0.1 | 127.0.0.1 | 0.053241 8   |
| Seq=4 Ack=16 Win=65280 Len=0 [ACK] 5566 → 62345 44                             | TCP | 127.0.0.1 | 127.0.0.1 | 0.053318 9   |
| Seq=4 Ack=16 Win=65280 Len=0 [ACK] 5566 → 62345 44                             | TCP | 127.0.0.1 | 127.0.0.1 | 11.795622 10 |
| Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN, ACK] 62348 → 5566 56    | TCP | 127.0.0.1 | 127.0.0.1 | 11.795834 11 |
| Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5566 → 62348 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 11.795899 12 |
| Seq=1 Ack=1 Win=65280 Len=7 [PSH, ACK] 5566 → 62348 51                         | TCP | 127.0.0.1 | 127.0.0.1 | 11.796283 13 |
| Seq=1 Ack=8 Win=65280 Len=0 [ACK] 62348 → 5566 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 11.796320 14 |
| Seq=1 Ack=8 Win=65280 Len=7 [PSH, ACK] 62348 → 5566 51                         | TCP | 127.0.0.1 | 127.0.0.1 | 11.797316 15 |
| Seq=8 Ack=8 Win=65280 Len=0 [ACK] 5566 → 62348 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 11.797374 16 |
| Seq=16 Ack=4 Win=65280 Len=16 [PSH, ACK] 62345 → 5566 60                       | TCP | 127.0.0.1 | 127.0.0.1 | 11.806856 17 |
| Seq=4 Ack=32 Win=65280 Len=0 [ACK] 5566 → 62345 44                             | TCP | 127.0.0.1 | 127.0.0.1 | 11.807831 18 |
| Seq=8 Ack=8 Win=65280 Len=16 [PSH, ACK] 62348 → 5566 60                        | TCP | 127.0.0.1 | 127.0.0.1 | 11.807243 19 |
| Seq=1 Ack=24 Win=65280 Len=0 [ACK] 5566 → 62348 44                             | TCP | 127.0.0.1 | 127.0.0.1 | 11.807321 20 |
| Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN, ACK] 62351 → 5566 56    | TCP | 127.0.0.1 | 127.0.0.1 | 20.267714 21 |
| Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5566 → 62351 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 20.267919 22 |
| Seq=1 Ack=1 Win=65280 Len=0 [PSH, ACK] 5566 → 62351 48                         | TCP | 127.0.0.1 | 127.0.0.1 | 20.267974 23 |
| Seq=1 Ack=5 Win=65280 Len=0 [ACK] 62351 → 5566 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 20.268143 24 |
| Seq=1 Ack=5 Win=65280 Len=7 [PSH, ACK] 62351 → 5566 51                         | TCP | 127.0.0.1 | 127.0.0.1 | 20.268174 25 |
| Seq=5 Ack=8 Win=65280 Len=0 [ACK] 5566 → 62351 44                              | TCP | 127.0.0.1 | 127.0.0.1 | 20.269322 26 |
| Seq=32 Ack=4 Win=65280 Len=21 [PSH, ACK] 62345 → 5566 65                       | TCP | 127.0.0.1 | 127.0.0.1 | 20.269391 27 |
| Seq=4 Ack=53 Win=65280 Len=0 [ACK] 5566 → 62345 44                             | TCP | 127.0.0.1 | 127.0.0.1 | 20.281006 28 |
| Seq=24 Ack=8 Win=65280 Len=21 [PSH, ACK] 62348 → 5566 65                       | TCP | 127.0.0.1 | 127.0.0.1 | 20.281092 29 |
| Seq=24 Ack=8 Win=65280 Len=21 [PSH, ACK] 62348 → 5566 65                       | TCP | 127.0.0.1 | 127.0.0.1 | 20.281238 30 |

כאן ניתן לראות בבירור ששם המשתמש שכבר קיים ניסה להתחבר ונתקל בREFUSED מצד השרת



א. שכבת הרשת (Network Layer - IP)

כתובות: התקשורת כולה מתבצעת בין Src: 127.0.0.1 ל-Dst: 127.0.0.1.

משמעות: שימוש ב-Loopback מעיד שהתהליכים רצים על אותה מכונה פיזית. המידע אינו יוצא לרשת החיצונית.

**ב. שכבת התעבורה (Transport Layer - TCP)**

פרוטוקול TCP נבחר בשל אמינותו (Reliability) והבטחת סדר ההודעות, הקריטיים לאפליקציית צ'אט.

## 1. הקמת הקשר (Three-Way Handshake):

- ניתן לראות את תהליך יצירת הקשר בשורות הראשונות של הלוג (שורות 12-14 בתמונה):
- SYN: הלקוח שולח בקשת התחברות לפורט 5566.
- SYN,ACK: השרת מאשר ומסנכרן
- ACK: הלקוח מאשר. החיבור במצב ESTABLISHED.

2. העברת נתונים (Data Transfer): ההבדל בין  $LEN=0$  ל- $LEN>0$ :

בניתוח הלוג ניתן להבחין בשני סוגים עיקריים של חבילות לאחר החיבור:

א. חבילות מידע (Push):

- מסומנות בדגלים [PSH, ACK].
- אורך המידע (Len) גדול מ-0.
- משמעות: אלו החבילות שמכילות את המידע האמיתי של הצ'אט (שמות משתמשים, תוכן ההודעות).
- דוגמה מהתמונה: בשורה 5, Len=11. זוהי הודעת ה-Login הראשונה שנשלחה (שם המשתמש היה באורך 5 תווים, למשל "user1"). בשורה 80, Len=11, זוהי הודעת צ'אט.

ב. חבילות אישור (Acknowledgement):

- מסומנות בדגל [ACK] בלבד.
- אורך המידע הוא  $Len=0$ .
- משמעות: פרוטוקול TCP דורש שעל כל מידע שנשלח, הצד השני יחזיר "אישור קבלה".  
חבילות אלו הן ריקות מתוכן (Payload) ותפקידן רק להגיד "קיבלתי את החבילה הקודמת".  
לכן ה- $Length$  שלהן הוא 0.

### 3. חילוץ תוכן ההודעה (Message Payload):

|         |        |           |        |            |              |    |     |           |           |            |     |
|---------|--------|-----------|--------|------------|--------------|----|-----|-----------|-----------|------------|-----|
| Seq=1   | Ack=7  | Win=65280 | Len=7  | [PSH, ACK] | 62354 → 5566 | 51 | TCP | 127.0.0.1 | 127.0.0.1 | 27.006367  | 39  |
| Seq=7   | Ack=8  | Win=65280 | Len=0  | [ACK]      | 5566 → 62354 | 44 | TCP | 127.0.0.1 | 127.0.0.1 | 27.006444  | 40  |
| Seq=8   | Ack=7  | Win=65280 | Len=28 | [PSH, ACK] | 62354 → 5566 | 72 | TCP | 127.0.0.1 | 127.0.0.1 | 27.012587  | 47  |
| Seq=7   | Ack=36 | Win=65280 | Len=0  | [ACK]      | 5566 → 62354 | 44 | TCP | 127.0.0.1 | 127.0.0.1 | 27.012642  | 48  |
| Seq=36  | Ack=7  | Win=65280 | Len=32 | [PSH, ACK] | 62354 → 5566 | 76 | TCP | 127.0.0.1 | 127.0.0.1 | 35.492020  | 62  |
| Seq=7   | Ack=36 | Win=65280 | Len=0  | [ACK]      | 5566 → 62354 | 44 | TCP | 127.0.0.1 | 127.0.0.1 | 35.492153  | 63  |
| Seq=7   | Ack=36 | Win=65280 | Len=0  | [ACK]      | 5566 → 62354 | 44 | TCP | 127.0.0.1 | 127.0.0.1 | 104.581118 | 70  |
| Seq=68  | Ack=7  | Win=65280 | Len=28 | [PSH, ACK] | 62354 → 5566 | 72 | TCP | 127.0.0.1 | 127.0.0.1 | 104.581229 | 71  |
| Seq=68  | Ack=7  | Win=65280 | Len=28 | [PSH, ACK] | 62354 → 5566 | 72 | TCP | 127.0.0.1 | 127.0.0.1 | 116.076575 | 136 |
| Seq=105 | Ack=7  | Win=65280 | Len=28 | [PSH, ACK] | 62354 → 5566 | 72 | TCP | 127.0.0.1 | 127.0.0.1 | 116.076657 | 137 |
| Seq=105 | Ack=7  | Win=65280 | Len=28 | [PSH, ACK] | 62354 → 5566 | 72 | TCP | 127.0.0.1 | 127.0.0.1 | 136.477835 | 143 |
| Seq=105 | Ack=7  | Win=65280 | Len=28 | [PSH, ACK] | 62354 → 5566 | 72 | TCP | 127.0.0.1 | 127.0.0.1 | 136.477894 | 144 |

Wireshark · Follow TCP Stream (tcp.stream eq 3) · group\_61305-2\_app\_capture\_2.pcap

hanita  
APPROVELIST|noa,abigail,teva,hanitaLIST|noa,abigail,teva,hanita,lea  
teva|hi good morning!  
MSG|teva|hi good morning how are you?LIST|abigail,teva,hanita,leaLIST|teva,hanita,lea

כדי לראות את תוכן הצ'אט הממשי ("ההודעה ששלחתי"), ביצענו פעולת Follow TCP Stream ב-Wireshark.

ניתן לראות כאן את כל הפעולות שקרו לאותו משתמש במהלך ההפעלה של האפליקציה.

#### 4. ניתוק לא תקין (RST Flag)

|                                                           |     |           |           |                |
|-----------------------------------------------------------|-----|-----------|-----------|----------------|
| Seq=68 Ack=28 Win=65280 Len=3/ [PSH, ACK] 62354 → 5566 81 | TCP | 127.0.0.1 | 127.0.0.1 | 116.0/65/5 136 |
| Seq=28 Ack=105 Win=65280 Len=0 [ACK] 5566 → 62354 44      | TCP | 127.0.0.1 | 127.0.0.1 | 116.076657 137 |
| Seq=4 Ack=113 Win=0 Len=0 [RST, ACK] 5566 → 62345 44      | TCP | 127.0.0.1 | 127.0.0.1 | 136.469914 138 |
| Seq=128 Ack=8 Win=65280 Len=28 [PSH, ACK] 62348 → 5566 72 | TCP | 127.0.0.1 | 127.0.0.1 | 136.476402 139 |

#### בשורה 165 (מסומנת באדום בתמונה), נצפה דגל [RST, ACK].

דגל Reset נשלח כאשר הקשר נקטע באופן פתאומי ולא מסודר, כאשר המשתמש ישר התנתק מבלי לעשות exit. השרת מזהה שהלקוח "נעלם" ושולח RST כדי לסגור את ה-Socket ולשחרר משאבים.

|                                                        |     |           |           |                |
|--------------------------------------------------------|-----|-----------|-----------|----------------|
| Seq=27 Ack=88 Win=65280 Len=0 [ACK] 5566 → 62356 44    | TCP | 127.0.0.1 | 127.0.0.1 | 160.156690 161 |
| Seq=8 Ack=9 Win=65280 Len=0 [FIN, ACK] 5566 → 55156 44 | TCP | 127.0.0.1 | 127.0.0.1 | 168.569210 162 |
| Seq=9 Ack=9 Win=65280 Len=0 [ACK] 55156 → 5566 44      | TCP | 127.0.0.1 | 127.0.0.1 | 168.569314 163 |
| Seq=164 Ack=40 Win=0 Len=0 [RST, ACK] 62351 → 5566 44  | TCP | 127.0.0.1 | 127.0.0.1 | 185.840732 164 |
| Seq=153 Ack=28 Win=0 Len=0 [RST, ACK] 62354 → 5566 44  | TCP | 127.0.0.1 | 127.0.0.1 | 185.840878 165 |
| Seq=88 Ack=27 Win=0 Len=0 [RST, ACK] 62356 → 5566 44   | TCP | 127.0.0.1 | 127.0.0.1 | 185.840966 166 |
| Seq=8 Ack=156 Win=0 Len=0 [RST, ACK] 5566 → 62348 44   | TCP | 127.0.0.1 | 127.0.0.1 | 198.099498 167 |

ניתן לראות כאן שיש התנתקות יחד של כל היוזרים זה נובע מכך שיש כפתור STOP SERVER שיש בו GUI של השרת

#### 5. תיאור שימוש בבינה מלאכותית (AI):

- במהלך הפיתוח נעשה שימוש במודל שפה (LLM) למטרות הבאות:
- שיפור ממשק המשתמש (GUI): סיוע בכתיבת קוד tkinter ליצירת הבעות המעוגלות ("Bubbles") ועיצוב ה-Dark Mode, דבר שדרש חישוב קואורדינטות מורכב על גבי Canvas.
- הבנת פרוטוקולים: הסבר על ההבדלים בין דגלי TCP ב-Wireshark (בפרט ההבדל בין חבילות מידע PSH לחבילות אישור ריקות ACK עם Len=0).
- דיבוג (Debugging): איתור בעיה שבה הממשק הגרפי קפא בעת המתנה להודעה, ופתרונה באמצעות שימוש ב-Threading נפרד להאזנה (receive\_messages).

# דוגמאות לפרומפטים ששימשו אותנו:

- "כיצד ניתן ליצור בועות הודעה (Chat Bubbles) עם פינות עגולות ב-Tkinter כמו ב-WhatsApp?"
- "איך למנוע התנגשות בין תהליכונים (Race Condition) כאשר שני לקוחות מנסים לעדכן את רשימת המחוברים בשרת במקביל?"